MobilityAPI

An OGC API – Moving Features server over MobilityDB

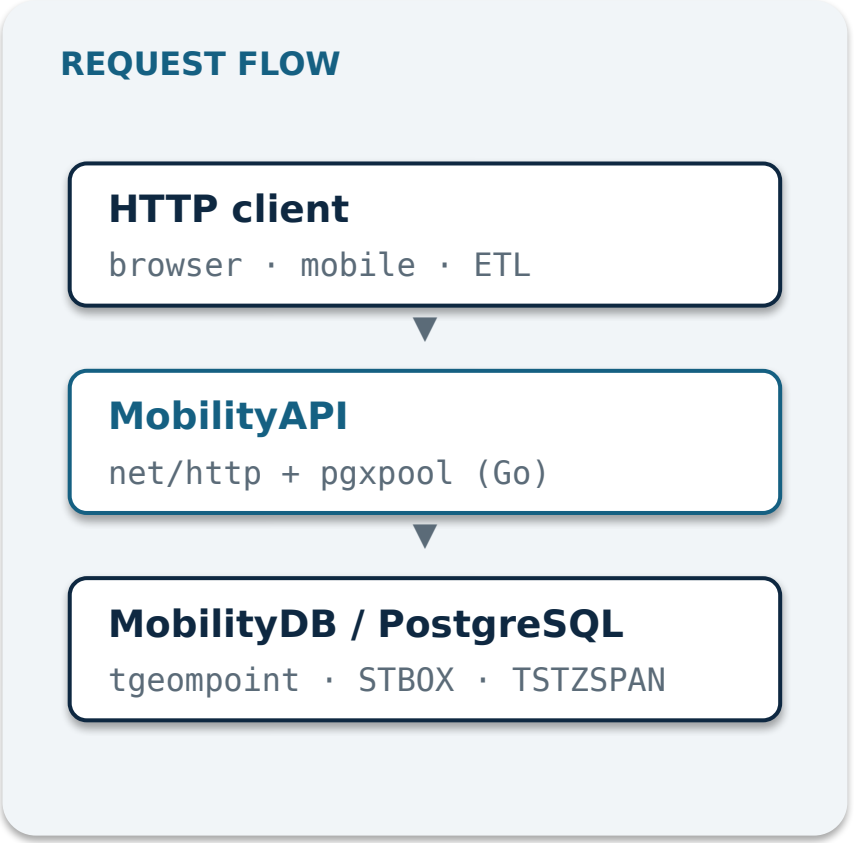
A thin compiled tier over MobilityDB · canonical Danish AIS example

net/http + pgxpool · single static binary · no MEOS in the tier

WHAT IT IS

A Thin Tier over MobilityDB, Fat Database

- **OGC-compliant** implements OGC API – Moving Features – Part 1: Core: collections, items, temporal geometry, derived queries, temporal properties
- **Thin compiled tier** net/http and a pgx connection pool; a single static binary, no cgo and no MEOS in the process
- **Fat database** every temporal computation stays in MobilityDB; the tier never parses or transforms a trajectory in memory
- **REST for any client** browsers, mobile, microservices and ETL consume mobility data without SQL or the PostgreSQL wire protocol
- **Bounded as data grows** streaming responses, keyset pagination and index-using filters keep memory and paging bounded as collections grow



ARCHITECTURE

Where the Work Happens

In the tier (Go)

- **Routing** net/http with the Go 1.22 pattern mux; one handler per OGC resource
- **Pooling** a pgx pool multiplexes requests onto a bounded set of database connections
- **Adapter** crs URN to EPSG and interpolation Stepwise to Step, roughly fifty lines
- **Streaming** responses are written row by row from a server-side cursor

In the database (MobilityDB)

- **Reads** asMFJSON assembles MovingFeaturesJSON in SQL; the tier streams the text
- **Writes** tgeompointFromMFJSON parses the payload into a tgeompoint
- **Filters** bbox and datetime push to the GiST index; atTime clips
- **Measures** speed, cumulativeLength and azimuth compute the derived properties

NO MEOS IN THE TIER

The trajectory is never materialised, parsed or transformed in the tier. It travels as asMFJSON text on the way out and as a MovingFeaturesJSON payload on the way in; MobilityDB does the parsing, the geometry algebra and the temporal computations. The result is a single static binary with no cgo dependency.

SAME DATA, TWO INTERFACES

MobilityDB SQL vs MobilityAPI REST

PostgreSQL / MobilityDB

```
INSERT INTO ships (id, mmsi, name, trip)
VALUES (
  205106000, 205106000, 'MARCUS',
  tgeompoint '[Point(645829 6058110)@2026-02-26
06:00+00,
              Point(651280 6058390)@2026-02-26
06:25+00,
              Point(656731 6058670)@2026-02-26
06:50+00]'
);
```

MobilityAPI

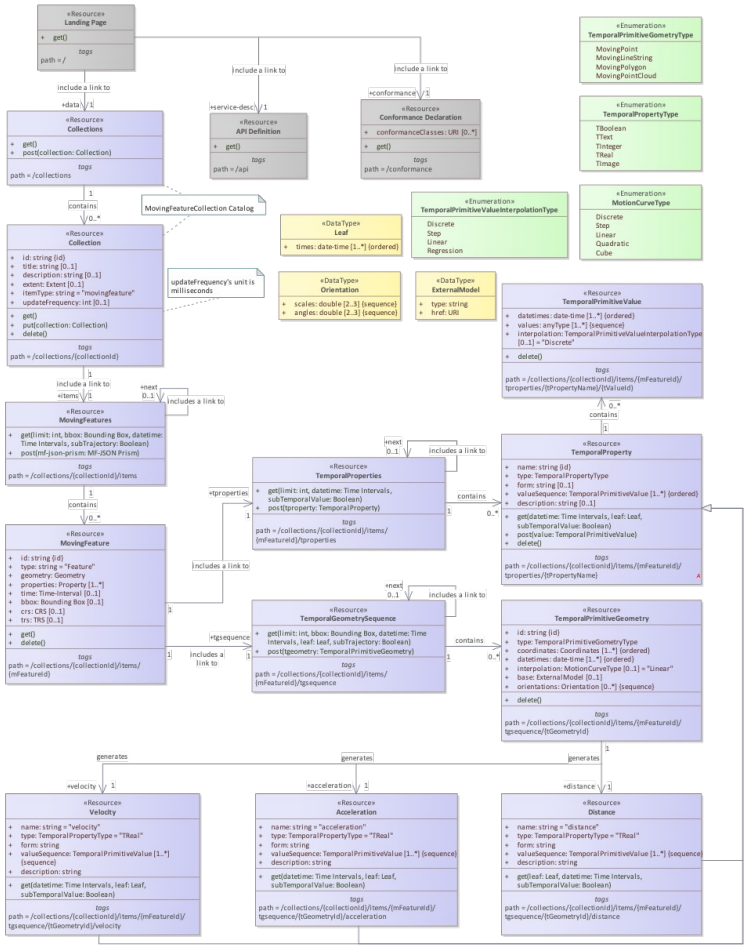
```
POST /collections/ships/items
Content-Type: application/json

{
  "type": "Feature",
  "id": "205106000",
  "properties": { "name": "MARCUS" },
  "temporalGeometry": {
    "type": "MovingPoint",
    "datetimes": ["2026-02-26T06:00:00+00", ...],
    "coordinates": [[645829, 6058110], ...],
    "interpolation": "Linear"
  }
}
```

Identical trajectory. The REST client never writes SQL or opens a database connection.

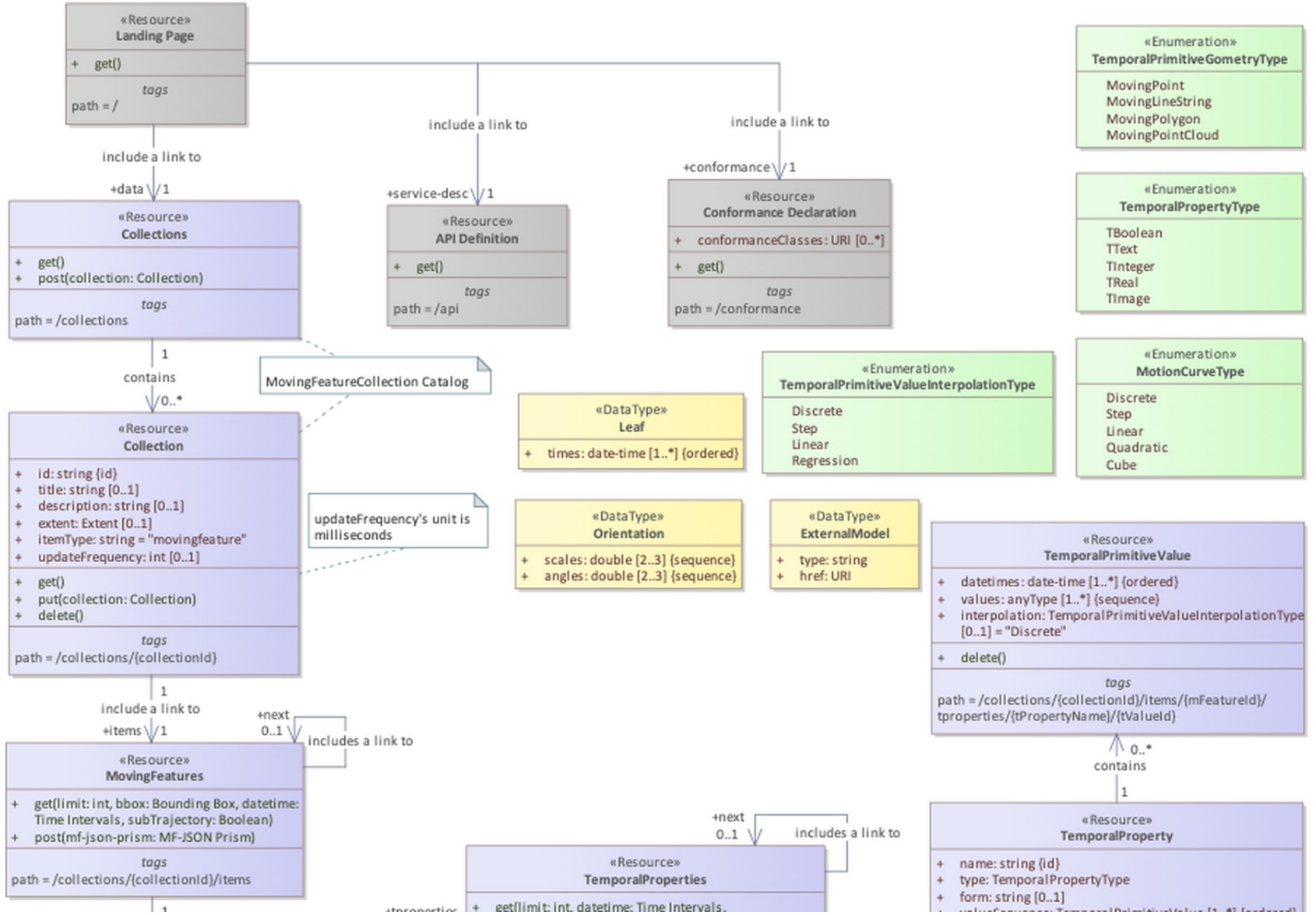
STANDARD

OGC API – Moving Features Data Model



Class diagram — OGC API – Moving Features – Part 1: Core (fig. 1)

- **Collection** a named group of moving features (itemType movingfeature)
- **MovingFeature** an item with id, properties, crs and trs
- **TemporalGeometry** a MovingPoint: parallel coordinates and datetimes plus interpolation
- **Derived queries** distance and velocity over a temporal geometry
- **TemporalProperties** time-varying attributes derived from the trajectory


«Resource»
Landing Page
+ get()
tags
path = /
include a link to
+data
1
include a link to
+service-desc
1
include a link to
+conformance
1
«Resource»
Collections
+ get()
+ post(collection: Collection)
tags
path = /collections
«Resource»
API Definition
+ get()
tags
path = /api
«Resource»
Conformance Declaration
+ conformanceClasses: URI [0..*]
+ get()
tags
path = /conformance
«Enumeration»
TemporalPrimitiveGometryType
MovingPoint
MovingLineString
MovingPolygon
MovingPointCloud
«Enumeration»
TemporalPropertyType
TBoolean
TText
TInteger
TReal
TImage
1
contains
0..*
MovingFeatureCollection Catalog
«Resource»
Collection
+ id: string {id}
+ title: string [0..1]
+ description: string [0..1]
+ extent: Extent [0..1]
+ itemType: string = "movingfeature"
+ updateFrequency: int [0..1]
+ get()
+ put(collection: Collection)
+ delete()
tags
path = /collections/{collectionId}
«DataType»
Leaf
+ times: date-time [1..*] {ordered}
«Enumeration»
TemporalPrimitiveValueInterpolationType
Discrete
Step
Linear
Regression
«Enumeration»
MotionCurveType
Discrete
Step
Linear
Quadratic
Cube
updateFrequency's unit is milliseconds
«DataType»
Orientation
+ scales: double [2..3] {sequence}
+ angles: double [2..3] {sequence}
«DataType»
ExternalModel
+ type: string
+ href: URI
«Resource»
TemporalPrimitiveValue
+ datetimes: date-time [1..*] {ordered}
+ values: anyType [1..*] {sequence}
+ interpolation: TemporalPrimitiveValueInterpolationType [0..1] = "Discrete"
+ delete()
tags
path = /collections/{collectionId}/items/{mFeatureId}/tproperties/{tPropertyName}/{tValueId}
1
include a link to
+items
1
+next
0..1
includes a link to
«Resource»
MovingFeatures
+ get(limit: int, bbox: Bounding Box, datetime: Time Intervals, subTrajectory: Boolean)
+ post(mf-json-prism: MF-JSON Prism)
tags
path = /collections/{collectionId}/items
0..*
contains
1
+next
0..1
includes a link to
«Resource»
TemporalProperties
+ get(limit: int, datetime: Time Intervals,
«Resource»
TemporalProperty
+ name: string {id}
+ type: TemporalPropertyType
+ form: string [0..1]


tags
path = /collections/{collectionId}/items
1
contains
0..*
«Resource»
MovingFeature
+ id: string {id}
+ type: string = "Feature"
+ geometry: Geometry
+ properties: Property [1..*]
+ time: Time-Interval [0..1]
+ bbox: Bounding Box [0..1]
+ crs: CRS [0..1]
+ trs: TRS [0..1]
+ get()
+ delete()
tags
path = /collections/{collectionId}/items/{mFeatureId}
includes a link to
+tproperties
1
«Resource»
TemporalProperties
+ get(limit: int, datetime: Time Intervals, subTemporalValue: Boolean)
+ post(tproperty: TemporalProperty)
tags
path = /collections/{collectionId}/items/{mFeatureId}/tproperties
contains
1
0..*
+ name: string {id}
+ type: TemporalPropertyType
+ form: string [0..1]
+ valueSequence: TemporalPrimitiveValue [1..*] {ordered}
+ description: string [0..1]
+ get(datetime: Time Intervals, leaf: Leaf, subTemporalValue: Boolean)
+ post(value: TemporalPrimitiveValue)
+ delete()
tags
path = /collections/{collectionId}/items/{mFeatureId}/tproperties/{tPropertyName}
+next
0..1
includes a link to
1
+tgsequence
includes a link to
«Resource»
TemporalGeometrySequence
+ get(limit: int, bbox: Bounding Box, datetime: Time Intervals, leaf: Leaf, subTrajectory: Boolean)
+ post(tgeometry: TemporalPrimitiveGeometry)
tags
path = /collections/{collectionId}/items/{mFeatureId}/tgsequence
contains
1
0..*
«Resource»
TemporalPrimitiveGeometry
+ id: string {id}
+ type: TemporalPrimitiveGometryType
+ coordinates: Coordinates [1..*] {ordered}
+ datetimes: date-time [1..*] {ordered}
+ interpolation: MotionCurveType [0..1] = "Linear"
+ base: ExternalModel [0..1]
+ orientations: Orientation [0..*] {sequence}
+ delete()
tags
path = /collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}
generates
generates
generates
1
+velocity
1
+acceleration
1
+distance
1
«Resource»
Velocity
+ name: string = "velocity"
+ type: TemporalPropertyType = "TReal"
+ form: string
+ valueSequence: TemporalPrimitiveValue [1..*] {sequence}
+ description: string
+ get(datetime: Time Intervals, leaf: Leaf, subTemporalValue: Boolean)
tags
path = /collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}/velocity
«Resource»
Acceleration
+ name: string = "acceleration"
+ type: TemporalPropertyType = "TReal"
+ form: string
+ valueSequence: TemporalPrimitiveValue [1..*] {sequence}
+ description: string
+ get(datetime: Time Intervals, leaf: Leaf, subTemporalValue: Boolean)
tags
path = /collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}/acceleration
«Resource»
Distance
+ name: string = "distance"
+ type: TemporalPropertyType = "TReal"
+ form: string
+ valueSequence: TemporalPrimitiveValue [1..*] {sequence}
+ description: string
+ get(leaf: Leaf, datetime: Time Intervals, subTemporalValue: Boolean)
tags
path = /collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}/distance

UNDER THE HOOD

How MobilityAPI Persists Moving Features

collections

PK **id**	TEXT
title	TEXT
description	TEXT
item_type	TEXT
crs	INTEGER

ships (one table per collection)

PK **id**	INTEGER
mmsi	BIGINT
name	TEXT
trip	tgeompoint

- **collections** a small registry: each row maps a collection id to its feature table and CRS
- **feature table** one row per moving feature; the trajectory is a single MobilityDB tgeompoint column
- **GiST index** gist (trip) backs the bbox and datetime filters with one spatio-temporal index
- **Derived properties** speed, distance and heading are computed from trip on demand, not stored

trip is a native MobilityDB type; the bounding box and time span come from its inline STBOX at O(1) cost.

CANONICAL EXAMPLE

A Day of Danish AIS Ship Traffic

- **Source** Danish Maritime Authority AIS feed (aisdata.ais.dk)
- **Window** 2026-02-26 to 2026-02-27
- **2,833 trajectories** from 1,878 distinct vessels (MMSI), each a MovingPoint
- **6.2 million instants** of observed position across the collection
- **CRS** EPSG:25832 (ETRS89 / UTM 32N, metres)

Danish Maritime Authority AIS vessel trajectories

REFERENCE

The Implemented Endpoint Surface

COLLECTIONS

- GET /collections
- GET /collections/{id}

MOVING FEATURES

- GET POST /collections/{id}/items
- GET PUT DELETE …/items/{fid}

TEMPORAL GEOMETRY

- GET POST …/items/{fid}/tgsequence
- GET …/tgsequence/{tgid}/distance · velocity

TEMPORAL PROPERTIES

- GET …/items/{fid}/tproperties
- GET …/tproperties/{name}

SERVICE

- GET /conformance · /api
- GET …/export (NDJSON · Parquet)

Filters on the items resource: bbox, datetime, subtrajectory, leaf, limit, after (keyset).

WALKTHROUGH

The Ships Collection, End to End

List → filter → retrieve → derive → write, all over HTTP

List Collections · Retrieve One

GET `/collections`

200 OK


```
{
  "collections": [
    {
      "id": "ships",
      "title": "ships",
      "itemType": "movingfeature",
      "description": "Danish AIS vessel trajectories",
      "links": [
        { "rel": "items",
          "href": "/collections/ships/items" },
        { "rel": "enclosure",
          "href": "/collections/ships/export",
          "type": "application/x-ndjson" }
      ]
    }
  ]
}
```


GET `/collections/ships`

200 OK


```
{
  "id": "ships",
  "title": "ships",
  "itemType": "movingfeature",
  "description": "Danish AIS vessel
                  trajectories
                  (MobilityDB tgeompoint)"
}
```


- itemType is movingfeature, per OGC API – Moving Features
- Links are relative; the client follows them without hard-coding paths

Stream the FeatureCollection, Page by Keyset

GET `/collections/ships/items?limit=2`

```
200 OK   Content-Type: application/json
{
  "type": "FeatureCollection",
  "features": [
    { "id": "1", "type": "Feature",
      "properties": { "mmsi": 111219510, "name": "vessel 111219510" },
      "temporalGeometry": {
        "type": "MovingPoint",
        "crs": { "type": "Name",
                 "properties": { "name":
"urn:ogc:def:crs:EPSG::25832" } },
        "sequences": [ { "datetimes": ["2026-02-26T13:28:08+01", ...],
                         "coordinates": [[721000.4, 6178001.2], ...] } ],
        "interpolation": "Linear" } },
    { "id": "2", "type": "Feature", ... }
  ],
  "numberReturned": 2,
  "links": [ { "rel": "next",
               "href": "/collections/ships/items?after=2&limit=2" } ]
}
```

- **Streamed** each Feature is written from a server-side cursor; memory is bounded for any page size
- **Keyset next** the next link advances by id (after=), with no OFFSET
- **asMFJSON** the temporalGeometry is assembled in SQL and streamed as text
- **limit** caps the page; the tier enforces a maximum

Retrieve a Single Vessel

GET `/collections/ships/items/1`

```
200 OK
{
  "id": "1",
  "type": "Feature",
  "properties": {
    "mmsi": 111219510,
    "name": "vessel 111219510"
  },
  "temporalGeometry": {
    "type": "MovingPoint",
    "crs": { "type": "Name", "properties": {
      "name": "urn:ogc:def:crs:EPSG::25832" } },
    "sequences": [
      { "datetimes": ["2026-02-26T13:28:08+01", ...],
        "coordinates": [[721000.4, 6178001.2], ...] } ],
    "interpolation": "Linear"
  }
}
```

- **One movement** the full trajectory of one vessel as MovingFeaturesJSON
- **crs** carried as an OGC URN; the tier maps to and from the EPSG form
- **interpolation** Linear: the vessel moves continuously between fixes
- **sequences** one per continuous run; gaps split the trajectory

SPATIAL FILTER

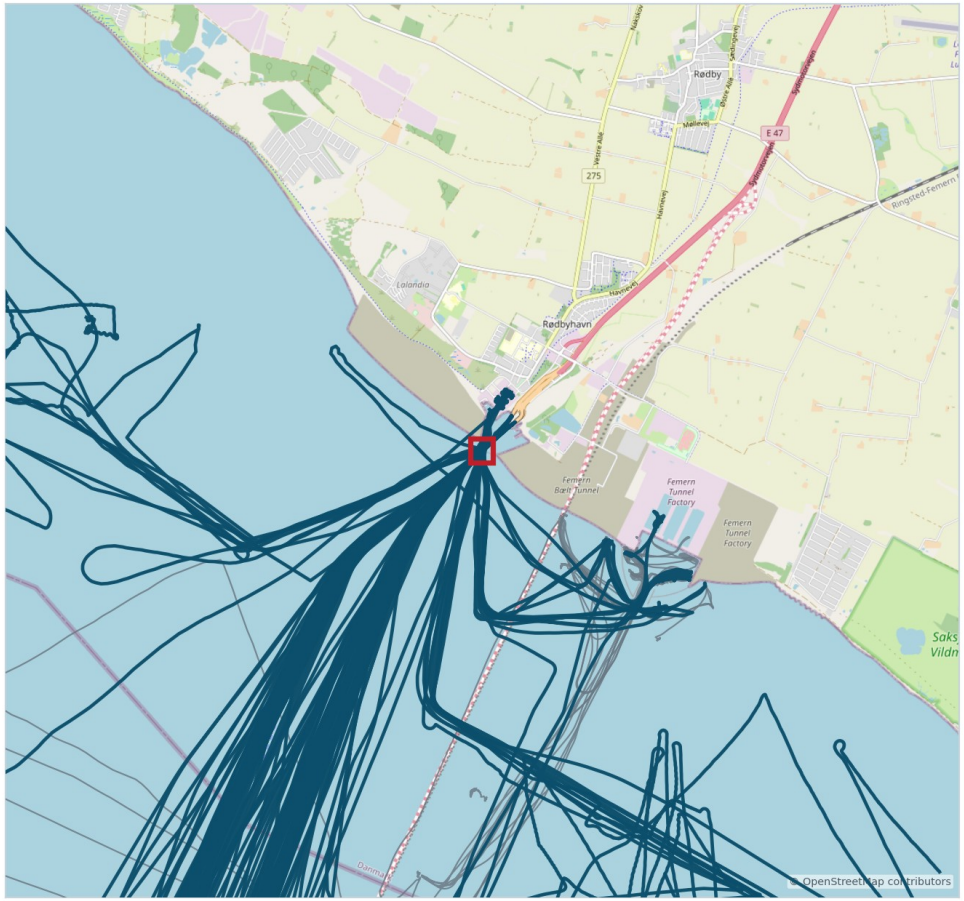
Vessels Intersecting a Bounding Box

GET `/collections/ships/items?bbox=…`

```
GET /collections/ships/items
      ?bbox=580000,6320000,600000,6340000
      &limit=10000

// bbox = minx,miny,maxx,maxy in the storage CRS
// EPSG:25832 (metres)
// 104 of 2,833 trajectories intersect the box
```

- **bbox** keeps vessels whose trajectory intersects the envelope
- **Index** pushed to the MobilityDB GiST index via the overlap operator
- **CRS** coordinates are in the collection's storage CRS (EPSG:25832)
- **Composable** with datetime, subtrajectory, limit and after



Trajectories crossing a bounding box (teal); context in grey

TEMPORAL SLICE

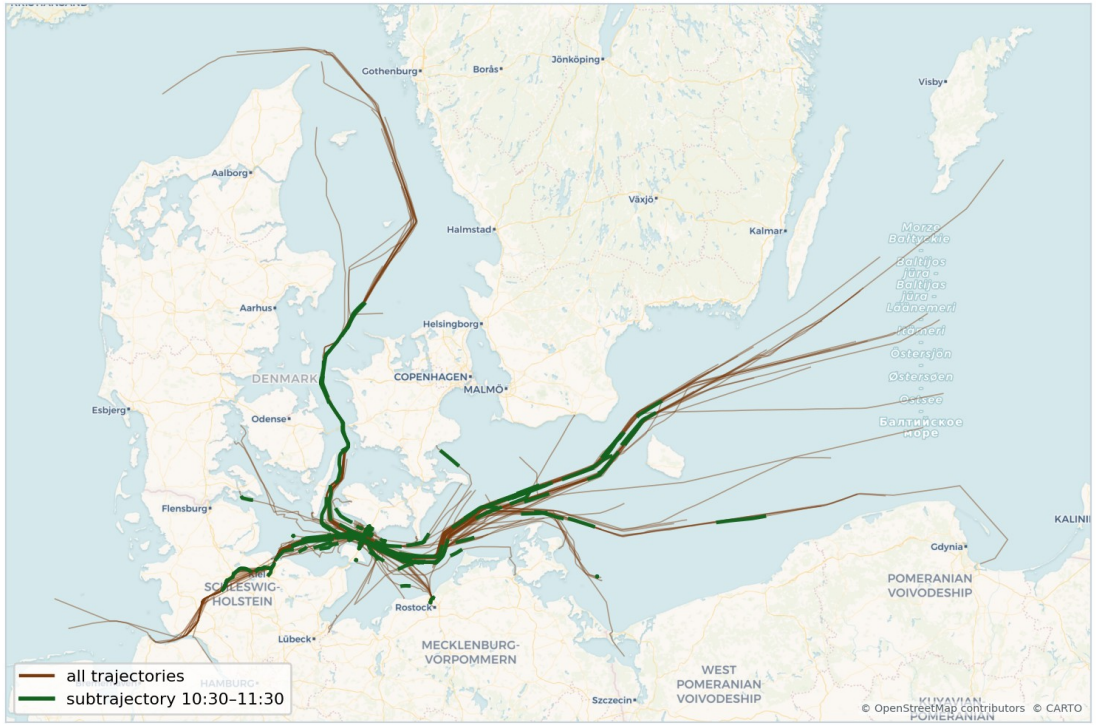
Restrict Trajectories to a Time Interval

GET **…/items?subtrajectory=true**

```
GET /collections/ships/items
      ?subtrajectory=true
      &datetime=2026-02-26T13:30:00+01
               /2026-02-26T13:35:00+01

// datetime is an RFC 3339 interval (start/end)
// subtrajectory=true clips each trajectory to the
// window with atTime, instead of filtering trips
```

- **subtrajectory** clips each trajectory to the datetime interval via atTime
- **datetime** RFC 3339 interval; a bare instant is also accepted
- **Gap aware** rows whose values fall entirely in a temporal gap are dropped
- **Validation** subtrajectory requires a bounded interval



Subtrajectories within a time window (green) over full trajectories (brown)

WRITES

Create · Replace · Append · Delete

POST `/collections/ships/items`

```
{ "type": "Feature", "id": "900001",
  "properties": { "name": "demo" },
  "temporalGeometry": {
    "type": "MovingPoint", "interpolation": "Linear",
    "datetimes": ["2026-02-26T10:00:00+01", ...],
    "coordinates": [[600000, 6330000], ...] } }

201  { "message": "created", "id": "900001" }
```

POST `…/items/900001/tgsequence`

```
// append a temporally-disjoint sub-trajectory
200  { "message": "appended", "id": "900001" }

// overlapping the existing time span:
409  { "code": "409",
       "description": "the sub-trajectory overlaps
        the existing temporal geometry in time" }
```

PUT `…/items/900001`

200 replace the feature

DELETE `…/items/900001`

204 remove the feature

- **merge** append uses MobilityDB merge, which rejects time overlap; the tier maps that to 409
- **Derived properties** are computed from the geometry, so writes to them return 501; modify the geometry instead
- **Single geometry** a feature carries one temporal geometry; delete the feature rather than a primitive

The Temporal-Geometry Sequence

GET `…/items/1/tgsequence`

```
200 OK
{
  "type": "MovingPoint",
  "crs": { "type": "Name", "properties": {
    "name": "urn:ogc:def:crs:EPSG::25832" } },
  "sequences": [
    { "datetimes": ["2026-02-26T13:28:08+01", ...],
      "coordinates": [[721000.4, 6178001.2], ...],
      "lower_inc": true, "upper_inc": true }
  ],
  "interpolation": "Linear"
}
```

Selectors on this resource

- **datetime** clip the sequence to an interval (atTime)
- **leaf** return the geometry at one or more instants
- **POST** append a temporally-disjoint sub-trajectory

The temporal geometry is the trajectory itself, returned as MovingFeaturesJSON straight from asMFJSON.

Distance · Velocity · Acceleration

GET …/tgsequence/0/velocity

```
200 OK
{ "name": "velocity", "type": "TReal", "form": "m/s",
  "description": "Speed over ground ...",
  "valueSequence": [
    { "datetimes": ["2026-02-26T13:28:08+01", ...],
      "values": [22.78, 11.03, ...],
      "interpolation": "Stepwise" } ],
  "links": [ { "rel": "self", ... } ] }
```

GET …/tgsequence/0/acceleration

```
501
{ "code": "501", "description":
  "acceleration is not derivable: linearly
   interpolated position gives a piecewise-constant
   speed, whose derivative is zero within each
   segment and undefined at the vertices" }
```

- **velocity** speed over ground from speed(trip), piecewise-constant (Stepwise)
- **distance** cumulative length from cumulativeLength(trip) (Linear)
- **acceleration** not fabricated: with linear position the speed is a step function, so its derivative is not a finite function
- **Exact** each measure is a MobilityDB function reshaped into an OGC temporalProperty (TReal)

Time-Varying Attributes per Feature

GET `…/items/1/tproperties`

```
200 OK
{
  "temporalProperties": [
    { "name": "velocity", "type": "TReal", "form":
"m/s" },
    { "name": "distance", "type": "TReal", "form": "m" },
    { "name": "heading",  "type": "TReal", "form":
"rad" }
  ],
  "numberReturned": 3,
  "links": [ { "rel": "self", ... } ]
}
```

GET `…/tproperties/velocity?leaf=…`

```
{ "name": "velocity", "type": "TReal", "form": "m/s",
  "valueSequence": [
    { "datetimes": ["2026-02-26T13:28:08+01"],
      "values": [22.78],
      "interpolation": "Discrete" } ] }
```

Property type

TBoolean · TText · TInteger · TReal · TImage

- **Derived** velocity, distance and heading are computed from the trajectory, read-only
- **valueSequence** datetimes and values with each segment's true interpolation
- **leaf** selects values at given instants; the result is Discrete
- **datetime** clips the property to an interval

CONFORMANCE

Declared Classes, API Definition, Errors

Conformance classes

- movingfeatures / conf / common
- / conf / mf-collection
- / conf / movingfeatures
- ogcapi-common-1 / conf / core
- ogcapi-common-2 / conf / collections

API definition

- **/api** an OpenAPI 3.0 definition
- **service-desc** linked from the landing page
- **Landing** self, service-desc, conformance, data
- Clients discover the surface at runtime

OGC error responses

- **400** invalid parameters or body
- **404** collection / feature not found
- **409** sub-trajectory overlaps in time
- **501** measure not derivable / derived write
- Body: { "code", "description" }

SCALE AND LAKEHOUSE

Bounded Memory at Any Result Size

393 MB

streamed in one NDJSON export

≈18 MB

resident memory, bounded by the stream and not the result size

- **Streaming responses** the FeatureCollection is written row by row from a server-side cursor, so memory is bounded for any result size
- **Keyset pagination** next links advance by id with no OFFSET, so paging stays constant cost through arbitrarily large collections
- **Index-using filters** bbox and datetime push to the MobilityDB GiST index; subtrajectory clips with atTime
- **Lakehouse export** NDJSON for direct ingestion, or Parquet carrying trajectory WKB plus a bbox and time sidecar; the row group bounds export memory

Every temporal computation stays in MobilityDB; the tier is a single static binary. A step toward the Data Lake House architecture in the MobilityDB ecosystem.

RECAP

Everything the Server Implements

COLLECTIONS

- ✓ list (GET)
- ✓ retrieve (GET)
- ✓ registry-backed

MOVING FEATURES

- ✓ insert (POST)
- ✓ retrieve / stream
- ✓ replace (PUT)
- ✓ delete (DELETE)
- ✓ keyset paging

QUERYING

- ✓ bbox spatial filter
- ✓ datetime interval / instant
- ✓ subtrajectory clipping
- ✓ leaf selector

TEMPORAL GEOMETRY

- ✓ get sequence (GET)
- ✓ append sub-trajectory (POST)
- ✓ datetime / leaf clip

DERIVED MEASURES

- ✓ distance
- ✓ velocity
- ✓ acceleration → 501 (not derivable)
- ✓ exact, as TReal

CONFORMANCE & SCALE

- ✓ 5 conformance classes
- ✓ OpenAPI at /api
- ✓ streaming + keyset
- ✓ NDJSON · Parquet export

Thanks for Listening

Questions?

REFERENCES

Find Out More

- **OGC API – Moving Features – Part 1: Core**
  https://docs.ogc.org/is/22-003r3/22-003r3.html

- **MobilityDB**
  https://github.com/MobilityDB/MobilityDB · https://mobilitydb.com

- **MEOS / ecosystem overview**
  https://libmeos.org

- **AIS data — Danish Maritime Authority**
  http://aisdata.ais.dk